

RAG Practice Manual

A synthetic multi-page manual for extraction, page metadata, cleaning and chunking exercises. Every page carries a running header and a numbered footer so that header and footer removal has something to detect.

1. Introduction

Retrieval-augmented generation puts a lookup step in front of a language model. The question is used to find relevant passages in a collection, those passages are placed into the prompt, and the model is asked to answer from them.

The point is to supply knowledge the model does not hold: material published after training, or private to an organisation, or too specific to have been memorised. The model supplies language ability while the collection supplies facts.

Compared with fine-tuning, the knowledge stays editable. Correcting an error means editing a document and re-indexing it, rather than assembling a dataset and running a training job.

2. The Pipelines

A RAG system has two pipelines that run at different times. Indexing happens ahead of a question: documents are loaded, cleaned, split into passages, encoded, and written to a store together with their metadata.

Querying happens per question: the question is encoded, the store returns the closest passages, those passages are assembled into a prompt, and the model produces an answer that is returned with its sources.

Keeping the two separate matters because they have different performance requirements. Indexing is a batch job that may take minutes, while querying sits in front of a user and is measured in hundreds of milliseconds.

3. Chunking

Fixed-Size Chunking

Fixed-size chunking walks the text and cuts at a set number of characters or tokens. The resulting pieces have predictable lengths, which makes capacity planning straightforward and keeps every piece comfortably inside a model's input limit.

The cost is that cuts land wherever the counter happens to reach the limit, frequently mid-sentence and occasionally mid-word. A piece that begins halfway through a clause can read as nonsense when it is later shown to a model as evidence.

The strategy remains a reasonable default for uniform material such as transcripts or logs, where paragraph structure carries little meaning and predictable sizing matters more than clean boundaries.

Overlap Between Chunks

Overlap repeats a tail of each piece at the head of the next one. A fact stated across a boundary then appears complete in at least one piece, instead of being halved and lost by both.

A common starting point is ten to twenty percent of the target size. Larger overlaps increase storage and cause the same sentence to surface several times in a result list, crowding out genuinely different material.

Overlap must be strictly smaller than the piece size. When the two are equal the window never advances and the loop that produces pieces runs forever, which is the most common bug in a hand-written splitter.

4. Embeddings

What an Embedding Is

An embedding maps a piece of text onto a fixed-length list of floating point numbers. Texts that a model judges to mean similar things land near one another in that space, while unrelated texts land far apart.

The mapping is learned, not designed. Nothing forces dimension 47 to correspond to a concept a human could name, and inspecting individual coordinates is rarely informative. What carries meaning is the geometric relationship between whole vectors.

Because the output length is fixed, a single word and a full paragraph both become vectors of the same size. This is what allows a short query and a long passage to be compared directly with one arithmetic operation.

Cosine Similarity

Cosine similarity measures the angle between two vectors and ignores their lengths. It is computed as the dot product divided by the product of the two magnitudes, and for embeddings it ranges in practice from roughly zero to one.

Ignoring magnitude is usually what is wanted, because a longer passage tends to produce a vector with a larger norm without being more relevant. Euclidean distance would let that length difference dominate the comparison.

Absolute scores are not comparable across different embedding models. One model may place unrelated sentences at 0.1 and another at 0.7, so any threshold has to be calibrated against the specific model in use rather than copied from a tutorial.

5. Vector Stores

What a Vector Database Does

A vector database stores numeric representations next to the text and metadata they were derived from, and answers the question "which stored items sit closest to this one" without comparing against every row.

Exhaustive comparison is exact but its cost rises in direct proportion to collection size, which becomes the dominant latency term somewhere in the tens of thousands of items. Approximate indexes trade a small amount of recall for a search time that grows far more slowly.

Alongside the vectors the store keeps an identifier, the original text, and arbitrary metadata fields. Returning the text with the match is what lets an application show a result rather than a row of numbers.

Working with Chroma Collections

Chroma organises data into collections, each of which holds documents, identifiers, optional metadata dictionaries, and the vectors themselves. Creating a client with a persistence path writes the collection to disk so it survives a restart.

Adding items requires an identifier per document. Reusing an existing identifier updates that entry rather than inserting a duplicate, which makes re-indexing a changed file straightforward as long as identifiers are derived deterministically from the source.

Queries accept either raw text, in which case the collection embeds it with its configured function, or a vector computed by the caller. A where clause filters on metadata before the similarity comparison, narrowing the candidate set rather than discarding results afterwards.

6. Retrieval

BM25 Lexical Ranking

BM25 scores a document against a query by summing a contribution for each query term the document contains. Terms that appear in few documents contribute more, because a rare word discriminates better than a common one.

Repetition helps with diminishing returns: the second occurrence of a term adds less than the first, and the curve saturates rather than growing without bound. A length correction stops long documents from scoring highly merely by containing more words.

The method matches surface forms and has no notion of meaning, so it is excellent on exact identifiers, product codes, and rare proper nouns, and blind to a question phrased entirely in synonyms of the document's vocabulary.

Semantic Retrieval

Semantic retrieval encodes the query with the same model used for the collection and ranks stored items by closeness in vector space. A question and a passage that never share a word can still rank first if the model places them near each other.

This handles paraphrase, synonymy, and indirect phrasing, which is exactly where term matching fails. It is the reason a question written in plain language can find a passage written in technical vocabulary.

The weakness is the mirror image: exact strings carry no special weight. A precise error code or version number may be outranked by a passage that discusses the same subject in general terms without containing the string at all.

7. Hybrid Search and Reranking

Hybrid Search

Hybrid search runs a term-based ranker and a vector ranker over the same collection and merges their outputs. Each compensates for the other's blind spot: exact strings come from the term side, paraphrase from the vector side.

A weighted blend needs the two score scales reconciled, because a BM25 score is unbounded while a cosine value sits near one. Normalising each result list to a common range before applying the weight is the usual fix.

The weight is a tunable parameter. Sweeping it from purely lexical to purely semantic on a labelled question set and plotting the metric shows where the corpus sits; identifier-heavy material favours the lexical end, conversational material the semantic end.

Cross-Encoder Reranking

A reranker reads the query and a candidate passage together and outputs a single relevance score. Because the two texts interact inside the model, it can judge fit far more precisely than a comparison of two independently computed vectors.

The cost is that nothing can be precomputed: a score exists only once a specific pair is evaluated, so the work is proportional to the number of candidates. Running it over a whole collection is impractical.

The standard arrangement fetches a few dozen candidates cheaply, reranks only those, and keeps the best handful. Recall is set by the first stage, so a passage the retriever missed can never be recovered by the reranker no matter how good it is.

8. Prompting and Grounding

Assembling the Augmented Prompt

The prompt given to the model normally carries an instruction, the retrieved passages each marked with an identifier, and the user's question. Separating the three with clear delimiters keeps the model from mistaking the question for part of the evidence.

The instruction should state explicitly that the answer must come from the supplied passages, and that the model is to say it does not know when they do not contain one. Without that sentence the model falls back on training knowledge and the citations stop meaning anything.

Ordering matters because models attend unevenly across a long context. Placing the strongest passage first, or repeating the question after the passages, measurably improves answers once the assembled context grows large.

Answers That Are Not Supported by the Sources

A language model will produce fluent, confident text whether or not the supplied passages justify it. The failure is not random noise; it reads exactly like a correct answer, which is what makes it dangerous.

The usual trigger is a question the passages do not answer. Given nothing relevant, the model falls back on patterns learned in training and fills the gap with something plausible. Instructing it to decline, and giving it an explicit way to do so, removes much of the pressure to invent.

Detection is a separate check: each claim in the output is compared against the passages that were actually provided, and anything unsupported is flagged. Requiring a citation per sentence makes that comparison mechanical.

9. Evaluation

Measuring Retrieval Quality

Recall at k asks whether a known correct source appears anywhere in the first k results. It is the ceiling on the whole system, because a passage that never reaches the generator cannot contribute to the answer.

Precision at k asks what fraction of those k results are relevant, which matters because irrelevant passages consume budget and distract the model. Mean reciprocal rank rewards placing the correct source early rather than merely including it.

Reporting recall and a rank-sensitive measure together is the informative pair. Recall alone hides a system that finds the right passage but buries it in tenth place.

Building an Evaluation Set

A usable evaluation set pairs each question with the identifiers of the sources that genuinely answer it. Without those labels no retrieval metric can be computed and changes can only be judged by impression.

Thirty to fifty questions is enough to detect a meaningful regression, provided they span the range of real usage: simple lookups, questions needing two sources, questions phrased in unexpected vocabulary, and questions the collection cannot answer at all.

That last category is the one most often omitted and the most informative, because it is the only way to observe whether the system declines to answer or invents something.

10. Operations and Security

Handling API Keys

A key committed to a repository is compromised the moment the repository is shared, and remains in the history after it is deleted from the current files. Rotating the key is the only real remedy once that has happened.

Keys belong in environment variables loaded from a file that version control ignores, or in a managed secret store that supplies them at runtime. The application should read the value at startup and fail loudly when it is absent.

Keys also leak through logs and error reports. Redacting anything that looks like a credential before writing a request to a log prevents a debugging aid from becoming a disclosure.

What to Log in a RAG System

The useful record for one request contains the original question, any rewritten forms, the identifiers and scores of the passages returned, the number of tokens consumed, the latency of each stage, and the final answer.

Stage timings separate a slow embedding call from a slow store from a slow generation, which is the first thing worth knowing when latency rises. A single total duration cannot distinguish them.

Retrieval scores logged over time reveal drift. A gradual fall in the top score across many questions usually means the corpus has moved away from what users are asking, well before anyone reports a bad answer.